

Filament Hotel Web Application

Technical Report

Assessment date: 29 August 2026 **Repository:** filament-hotel **Platform:** Laravel 12 / PHP 8.4 / Filament 5.2 **Assessment scope:** application source, routes, persistence layer, Filament administration, public Blade views, integrations, tests, and deployment configuration.

1. Executive summary

Filament Hotel is a Laravel-based hotel and restaurant operations system with a public guest website and a role-aware Filament administration panel. Its functional scope is broad: hotel-room booking, conference-room booking, restaurant table reservations, menu browsing and food ordering, Paystack payments, corporate credit billing, kitchen production and stock, reporting dashboards, content/gallery management, activity logs, and hotel branding.

The application has a solid foundation and a comparatively strong regression suite. The current test run completed with **125 tests and 601 assertions passing**. The main technical opportunities are maintainability and production hardening rather than a single blocking defect: several high-traffic workflows are still defined in large route files, deployment defaults are development-oriented, database/status conventions could be made more explicit, and operational concerns such as queue workers, scheduled commands, backups, and observability need documented production procedures.

2. Current-state inventory

Area	Current inventory
Registered Laravel routes	197
Eloquent models	28
HTTP controllers	20
Application services	10
Filament resources	25
Filament pages	21
Filament widgets	45
Database migrations	95
Database seeders	11
View files	96

Area	Current inventory
Test files	53
Physical authored source lines	38,564
Non-blank source lines	32,824

The line count includes PHP, Blade, JavaScript/TypeScript, CSS/SCSS, and Vue source files while excluding vendor, node_modules, storage, .git, and generated bootstrap/cache files.

3. Architecture

3.1 Application layers

- **HTTP and presentation:** Laravel routes, controllers, Blade templates, layouts, Tailwind CSS, Alpine.js, Vite, FullCalendar, and Flatpickr.
- **Domain and persistence:** Eloquent models represent guests, users, rooms, room types, conference facilities/rooms, restaurants, tables, menu categories/items, orders, reservations, payments, promotions, billing settings, corporate organizations, ingredients, stock movements, and kitchen production.
- **Business services:** billing calculation, corporate credit/payment settlement, invoices, room assignment, restaurant cart/kitchen behavior, kitchen stock, kitchen production reporting, and payment-report filtering are separated into service classes.
- **Administration:** one Filament panel is mounted at /admin; resource authorization is deliberately fail-closed through SecureResource, and role-specific dashboards are selected by the role dashboard entry point.

3.2 Dependencies and integrations

The primary dependencies are Laravel 12, Filament 5.2, Spatie Permission, Spatie Activitylog, Paystack HTTP integration, Dompdf invoice/PDF generation, and Simple QR Code. Front-end dependencies include Tailwind, Alpine, FullCalendar, Flatpickr, Axios, and Vite.

3.3 Routing organization

Routes are split into `auth.php`, `bookings.php`, `conference.php`, `guest.php`, `public.php`, `restaurant.php`, and `web.php`. The split is a useful improvement over a single route file, but `bookings.php`, `conference.php`, and `restaurant.php` still contain substantial inline closures and workflow logic. Those closures are harder to unit test, reuse, authorize, and instrument than dedicated controllers.

4. Functional capabilities

Guest-facing capabilities

- 1 Browse room types, availability, calendars, galleries, conferences, restaurant facilities, menu items, and restaurant tables.
- 2 Create and manage hotel bookings, conference bookings, table reservations, and food orders.
- 3 Choose immediate Paystack payment or corporate billing where the guest is linked to an enabled corporate organization.
- 4 View confirmations, invoices, payment state, outstanding corporate charges, profile information, and cart/order history from the guest dashboard.
- 5 Use QR/table ordering workflows for restaurant service.

Administration and operations

- Role dashboards for super administrators, administrators, accountants, managers, reception, kitchen managers, kitchen staff, and transaction reporting.
- Resources for guests, users, rooms, room types, conference rooms/facilities, restaurants, tables, menu categories/items, orders, reservations, payments, promotions, billing settings, hotel settings, corporate organizations, content, activity logs, ingredients, kitchen stock movements, and kitchen production.
- Booking calendar, room calendar/timeline, corporate receivables, occupancy, revenue, guest, restaurant, kitchen, and transaction reports.
- Payment actions and offline settlement support for corporate receivables.

Kitchen and inventory

Kitchen production supports batch output and ingredient consumption. Stock movement records provide auditable receipts, wastage, adjustments, and order/production consumption. Recipes connect menu items to ingredients, allowing finished portions and raw-ingredient movements to be tracked together.

5. Data and transaction model

The application uses 95 incremental migrations. A payment record can be linked to a hotel booking, conference booking, restaurant reservation, or restaurant order, with a guest reference where available. This supports a unified Payments page and transaction reporting, but it also means all four workflows must keep foreign keys, status values, and guest resolution rules consistent.

Billing calculations are centralized around configurable VAT, NHIL, service charge, and promotion/discount values. Corporate transactions remain unpaid until an accountant or authorized administrator records settlement. The application also includes hold-expiry/release commands so dates, slots, and rooms are not blocked indefinitely by abandoned checkouts.

6. Security and reliability assessment

Existing controls

- Laravel session authentication, CSRF protection, password confirmation, email verification, and login throttling.

- Filament panel authentication plus role-based access checks.
- Fail-closed Filament resource authorization through SecureResource.
- Signed/throttled verification routes and a named rate limiter for public restaurant reservations.
- Paystack webhook signature verification and idempotency checks in the food-order payment flow.
- Activity logging with password/remember-token exclusion tests and HTML escaping coverage.
- Image-upload validation and consistent public visibility for Filament uploads.

Risks and recommended hardening

- 1 **Production configuration:** `.env.example` uses `APP_DEBUG=true`, a local URL, SQLite, log mail, and development-style defaults. Production deployments should explicitly set `APP_ENV=production`, disable debug, use MariaDB/MySQL with TLS, configure real mail, use a durable object/file store, and rotate secrets.
- 2 **Route-level business logic:** extract large booking, conference, and restaurant closures into controllers and form requests. This will make authorization, validation, transactions, and error handling easier to review.
- 3 **Status/method consistency:** use PHP backed enums or shared constants for payment methods and payment/booking statuses. Database enum changes have previously caused truncation failures; a single domain vocabulary would reduce recurrence.
- 4 **Availability concurrency:** keep availability checks inside database transactions and add appropriate indexes/constraints. Application-level "check then create" logic can race under simultaneous bookings.
- 5 **Webhook and queue operations:** document Paystack webhook configuration, queue workers, retry policy, failed-job monitoring, and scheduler execution. The scheduler currently releases expired holds every minute and sends restaurant reminders daily.
- 6 **Backups and recovery:** define automated database/file backups, retention, restore drills, and migration rollback procedures, especially for payments, corporate receivables, and stock ledgers.
- 7 **Authorization breadth:** continue reviewing inline guest/accountant/admin actions with policies or dedicated authorization services, particularly around corporate account linkage and offline payment settlement.
- 8 **Observability:** add a production error tracker, structured request/payment identifiers, health checks, and alerts for webhook failures, queue backlog, failed jobs, and expired holds.
- 9 **Documentation:** replace the stock Laravel README with environment setup, migration/seed order, role matrix, Paystack setup, scheduler/queue requirements, and release/runbook documentation.

7. Quality assessment

The current suite is a meaningful strength. It covers authentication, authorization, booking availability/workflows, corporate deferred payment, payment webhooks, reports, dashboards, kitchen stock/production, public pages, uploads, and security regressions. The full run passed 125 tests and 601 assertions.

The suite is primarily PHPUnit feature coverage and source-level widget/view assertions. Recommended additions are browser-level smoke tests for the four checkout flows, Paystack sandbox contract tests, concurrency tests for overlapping availability, and migration tests against the production MariaDB version rather than only the test database.

8. Prioritized improvement roadmap

Priority 0 - production safety

- Lock down production environment variables and secret handling.
- Configure HTTPS, Paystack webhook endpoint, queue worker, scheduler, mail, backups, and monitoring.
- Run migrations against a staging clone and verify restore procedures.

Priority 1 - maintainability and correctness

- Move route closures into controllers and request classes.
- Introduce shared status/payment enums and database indexes for availability and transaction foreign keys.
- Add end-to-end checkout coverage for hotel, conference, table, and food-order flows.
- Add explicit idempotency keys and reconciliation reporting for every payment workflow.

Priority 2 - operational maturity

- Add structured logs, tracing/request IDs, health endpoints, and alerting.
- Improve README/runbooks and document role capabilities and data ownership.
- Profile report queries and add indexes/materialized summaries if production data volume grows.
- Add scheduled data-integrity checks for orphaned payments, stale holds, and stock ledger imbalance.

9. Conclusion

Filament Hotel is a feature-rich hospitality platform with a strong current baseline and broad operational coverage. The most valuable next step is production hardening and consolidation: make deployment behavior explicit, move workflow logic into testable classes, standardize statuses and payment reconciliation, and add browser/concurrency coverage around the highest-value paths. Those changes will reduce operational risk without requiring a redesign of the existing domain model or Filament panel.

Appendix A - Verification notes

- Full application test command: `php artisan test --stop-on-failure`
- Result at assessment time: **125 passed, 601 assertions.**
- The local assessment environment emitted a read-only warning while PHPUnit attempted to update `.phpunit.result.cache`; this did not affect test results.
- Migration status could not be confirmed in the read-only assessment environment because Laravel could not append to `storage/logs/laravel.log`. Verify migration state against the deployment database before release.